

Technical Report

Local RAG PDF Assistant

Mario Tuset

1. Problem Description

Accessing specific information from long documents such as PDFs or research papers can be difficult and is a time-consuming task. This application implements a local Question & Answering system based on RAG architecture that provides a graphical interface where users can upload their PDFs and ask questions to retrieve information from them. The system autonomously retrieves the most relevant text fragments and sends them to a local LLM, generating answers grounded in the provided document.

System desing and workflow

The system follows a multi-step pipeline that it is divided into two phases:

- Indexing Phase: The user uploads a PDF via the Streamlit sidebar and the PyPDFLoader extracts the text from the document. Then, RecursiveCharacterTextSplitter divides the text into chunks, where each chunk is vectorized using nomic-embed-text via Ollama. All obtained vectors are stored in a local FAISS index.
- Retrieval and Generation Phase: The user types a question in the Streamlit text input and it is vectorized using the same embedding model. FAISS performs a semantic similarity search and retrieves the top-k most relevant chunks and they are injected into a prompt template alongside the user's question. The local LLM generates the response and it is displayed in the Streamlit interface.

Model Selection

A two-model architecture was chosen, separating the retrieval and the generation tasks.

- LLama 3 (8B): Smaller models like Gemma 1B were discarded because RAG demands strong capabilities, where the model must synthesize disconnected text fragments and respond only based on them. Llama 3 offers the best balanced between local hardware viability and reasoning performance, making it a good

option for local RAG.

- **Nomic-embed-text**: This model was chosen because it is a dedicated embedding model trained for semantic search and retrieval task with a large context windows, allowing the application to capture the meaning of document chunk. Using a purpose-built embedding model is more efficient than repurposing a generative LLM for this task.

Implemenation Details

The application was built in Python combining useful libraries, each handling a specific part of the pipeline:

Streamlit provides the graphical interface where the user can upload its PDF, with the included sidebar, and ask the app a question with the chat input. It was chosen for its simplicity and rapid implementation.

LangChain connects the PDF loader, text splitter, embedding model, vector store and LLM into a single pipeline.

Ollama runs both AI models locally without requiring any cloud connection.

FAISS stores the document chunks as vectors and retrieves the most relevant one for later user's questions.

PyPDFLoader and **RecursiveCharacterTextSplitter** handle document preprocessing. The splitter uses chunks of 1000 characters with an overlap of 200 to avoid losing context.

Temporary files are deleted automatically after each PDF is processed. Cache resource ensures models are only loaded once at startup.

Discussion of Results, Limitations and Possible Improvements

The system successfully answers the user's questions grounded in the uploaded PDF. It was tested with an academic document divided into 19 chunks. The model explicitly references the provided context in its answers confirming that the retrieval is working correctly and the LLM is not relying on prior knowledge.

Referring to the limitations, only one document can be loaded at the time and response speed depends on local hardware because the models run locally. Also, PyPDFLoader cannot process scanned image-based PDFs.

Some possible improvements can be adding multi-turn conversation history for a proper chat experience or integrating OCR support to handle scanned images. Also, support multiple simultaneous document uploads can be a good improvement for the system.

Screenshots

Here is an example of the app working with a Bologna Process university assingment PDF uploaded and the user's query.

System response to the query "What was the Bologna Process?":

